1. Introduction

A GPS or navigation system helps users find the shortest or fastest route between two locations. Applications such as Google Maps use road-network data, traffic information, and routing algorithms to calculate routes.

A city map can be represented as a **weighted graph**, where intersections or locations are vertices, roads are edges, and edge weights represent distances or travel times.

Graph algorithms such as Dijkstra's algorithm are used to find the shortest path between locations.

2. Problem Statement

A city contains several locations connected by roads of different distances. A navigation system must determine the shortest route between a starting location and a destination.

The objective is to represent the city as a weighted graph and apply a graph algorithm to calculate the minimum-distance route.

3. Objectives

- To represent a city map using a weighted graph.
- To understand vertices, edges, and edge weights.
- To implement Dijkstra's shortest-path algorithm.
- To calculate the shortest distance between two locations.
- To analyze the time and space complexity of the algorithm.

4. Proposed System

The city is represented by a weighted, undirected graph.

- **Vertices:** Locations or road intersections.
- **Edges:** Roads connecting locations.
- **Weights:** Distances between connected locations.
- **Source:** Starting location.
- **Destination:** Target location.

For this example, road distances are measured in kilometres.

5. Example Dataset

Consider a city with five locations:

Road	Location 1	Location 2	Distance
Road 1	A	B	4 km
Road 2	A	C	2 km
Road 3	B	C	1 km
Road 4	B	D	5 km
Road 5	C	D	8 km
Road 6	C	E	10 km
Road 7	D	E	2 km

The objective is to find the shortest route from location A to location E.

6. Graph Representation

The road network can be represented as an adjacency list:

```
A -> B(4), C(2)
```

```
B -> A(4), C(1), D(5)
C -> A(2), B(1), D(8), E(10)
D -> B(5), C(8), E(2)
E -> C(10), D(2)
```
Each entry contains a neighbouring location and the distance to it.

7. Algorithm: Dijkstra's Shortest-Path Algorithm

1. Initialize the source distance to zero and all other distances to infinity.
2. Place the source location in a priority queue.
3. Remove the location with the smallest known distance.
4. Examine each neighbouring location.
5. If a shorter route is found, update its distance and predecessor.
6. Repeat until the destination is reached or no reachable locations remain.
7. Reconstruct the shortest path using predecessor information.
8. Display the route and total distance.

Dijkstra's algorithm works with non-negative edge weights.

8. Python Implementation

```
import heapq

graph = {
    'A': [('B', 4), ('C', 2)],
    'B': [('A', 4), ('C', 1), ('D', 5)],
    'C': [('A', 2), ('B', 1), ('D', 8), ('E', 10)],
    'D': [('B', 5), ('C', 8), ('E', 2)],
    'E': [('C', 10), ('D', 2)]
}

def dijkstra(graph, start, destination):
    distances = {node: float('inf') for node in graph}
    previous = {node: None for node in graph}

    distances[start] = 0
    pq = [(0, start)]

    while pq:
        current_distance, current = heapq.heappop(pq)

        if current_distance != distances[current]:
            continue

        if current == destination:
            break

        for neighbour, weight in graph[current]:
            new_distance = current_distance + weight

            if new_distance < distances[neighbour]:
                distances[neighbour] = new_distance
                previous[neighbour] = current
                heapq.heappush(
                    pq, (new_distance, neighbour)
                )

    if distances[destination] == float('inf'):
        return None, float('inf')

    path = []
    current = destination
```

```
    while current is not None:
        path.append(current)
        current = previous[current]

    path.reverse()
    return path, distances[destination]


path, distance = dijkstra(graph, 'A', 'E')

if path is None:
    print("Destination is unreachable")
else:
    print("Shortest Route:", " -> ".join(path))
    print("Total Distance:", distance, "km")
```

9. Sample Output

```
Shortest Route: A -> C -> B -> D -> E
Total Distance: 10 km
```

The route is calculated as follows:

- A to C = 2 km
- C to B = 1 km
- B to D = 5 km
- D to E = 2 km

Total distance = 2 + 1 + 5 + 2 = **10 km**.

10. Time and Space Complexity

Let (V) be the number of vertices and (E) the number of edges.

- Time complexity using an adjacency list and binary heap: (O((V+E)\log V)).
- Space complexity: (O(V+E)) for the graph, distance information, predecessors, and priority queue in the standard implementation.

11. Advantages

- Finds shortest paths for non-negative edge weights.
- Uses an efficient priority queue.
- Works well with sparse road networks.
- Can be adapted to calculate distance-based routes.
- Supports route planning between many locations.

12. Limitations

- Basic Dijkstra's algorithm does not support negative edge weights.
- Shortest distance may not be the fastest route when traffic varies.
- Real navigation requires updated road conditions, road closures, and travel-time estimates.
- Large road networks may require advanced routing techniques.

13. Applications

- GPS navigation systems.
- Delivery and logistics route planning.
- Emergency vehicle routing.
- Public transportation planning.
- Network routing.
- Location-based travel applications.

14. Conclusion

A GPS navigation system can model a city as a weighted graph and use Dijkstra's algorithm to calculate the shortest route between two locations. This approach efficiently determines routes when road weights are non-negative. Real-world systems extend this basic idea with live traffic data, road restrictions, and advanced routing algorithms.